

Plan de trabajo — Proyecto Final VoLTE (Team-C)

Simulación de "VoLTE UE Attachment and IMS Registration" (GSMA VoLTE Service Description and Implementation Guide V1.1, §3.2.1).

Equipo C — creado 2026-06-11 · Lenguaje: Python 3 · Transporte: UDP · Despliegue: cada integrante corre sus nodos en su laptop, sobre la LAN.

Documento maestro del proyecto. Los contratos detallados están en [docs/](#). Si editás este archivo, regenerará el PDF con [scripts/build_pdf.ps1](#).

1. Resumen ejecutivo

Tenemos que **simular una red VoLTE** en la que **12 nodos** (procesos) intercambian los mensajes de señalización de la **Figure 3** del enunciado, para lograr que **dos VoLTE UE** se **enganchen a la red móvil (Attach)** y se **registren en el IMS (Registration)**.

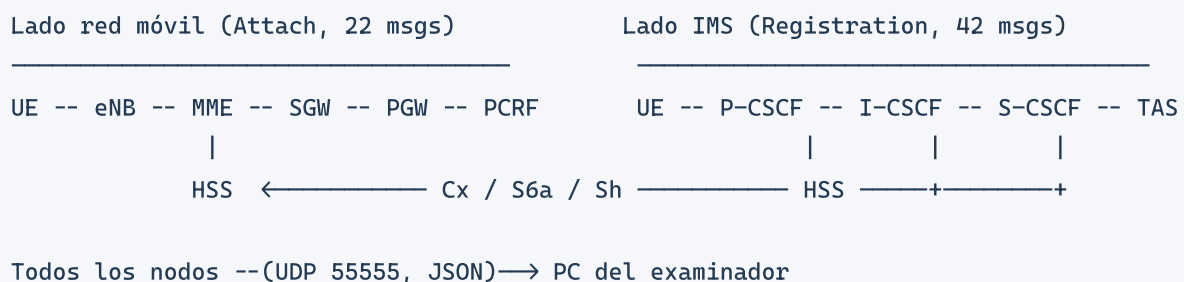
Cada integrante de Team-C (6 personas) **opera 2 nodos**. Cada nodo es un programa Python que:

1. **Escucha** mensajes de sus nodos vecinos por **UDP**.
2. **Procesa** cada mensaje según la máquina de estados (la Figure 3) y **reenvía** el mensaje siguiente al vecino que corresponda.
3. **Notifica al PC del examinador** (UDP 55555, JSON) **dos veces por cada mensaje recibido**: una al **recibir** (1 s después) y otra al **estar por enviar** el siguiente (1 s después, antes de mandarlo).

Al final, el **TAS** reporta los UE registrados y el **S-CSCF** manda `{"message": "fin"}`.

El éxito se mide por las notificaciones que ve el examinador, no por una "llamada" real: no hay voz ni cripto real; modelamos los mensajes y su orden.

2. Qué hay que construir



- **12 instancias de nodo:** **VoLTE UE 1**, **VoLTE UE 2**, **eNB**, **MME**, **SGW**, **PGW**, **PCRF**, **HSS**, **P-CSCF**, **I-CSCF**, **S-CSCF**, **TAS**.
- **Una librería compartida** ([common/](#)) que resuelve transporte UDP, formato de mensajes, notificaciones y el timing de 1 s. Todos la usan → todos notifican igual.
- **Un registro de direcciones** ([config/nodes.json](#)) con la IP y puerto de cada nodo y del examinador, más el apellido del operador de cada nodo.

Las tres cosas que **deben ser idénticas en los 12 nodos** (de ahí la librería común): el **envelope** de mensaje, el **JSON de notificación** y la **regla de timing**. Están especificadas en [docs/02_contratos.md](#).

3. Decisiones técnicas tomadas

Tema	Decisión	Por qué
Lenguaje	Python 3	Sockets UDP, JSON y timers triviales; multiplataforma; rápido de coordinar entre 6.
Transporte entre nodos	UDP	Mismo modelo que las notificaciones (UDP 55555); simple, sin conexiones.
Despliegue	LAN, una laptop por persona	Cada quien corre sus 2 nodos; direcciones por <code>config/nodes.json</code> . Se puede probar todo en localhost (puertos ya distintos).
Notificaciones	UDP 55555, JSON, timing 1 s + 1 s	Lo fija el enunciado.
Ejecución de 2 UE	Secuencial (UE1 completo, luego UE2)	Más simple y robusto; confirmar con examinador si exige paralelo.

4. Reparto de nodos (propuesta, ajustable)

12 instancias / 6 personas = **2 nodos cada uno**, balanceado por carga (Recibe+Emite). Detalle y fichas en [docs/03_asignacion_nodos.md](#).

Par	Nodos	Carga	Integrante (completar)
A	S-CSCF + PCRF	26	___
B	P-CSCF + PGW	25	___
C	HSS + I-CSCF	25	___
D	MME + eNB	24	___
E	VoLTE UE 2 + TAS	23	___
F	VoLTE UE 1 + SGW	21	___

Acción inmediata: en el kickoff, completar la última columna con los 6 nombres (Abreu, Birenbaum, Ferreira, Guasch, Roure, Silva) y poner cada apellido en `config/nodes.json`.

5. Plan por fases

Fase 0 — Kickoff y acuerdos (*todos, ~1–2 h, día 1*)

- Leer juntos [docs/00_consigna.md](#) y este plan.
- Confirmar decisiones técnicas (sección 3) y **repartir los 12 nodos** (sección 4).
- Designar **1 integrador** (mantiene `common/`, `config/nodes.json` y coordina las integraciones).
- **DoD:** reparto cerrado, integrador designado, todos con el repo clonado y Python 3 + el paquete `markdown` instalados.

Fase 1 — Esqueleto común + secuencia definitiva (*integrador + 1, días 1–2*)

- Implementar `common/` (config, transporte UDP, notificaciones con timing, clase `Node`) según `common/README.md`.
- Cerrar la **secuencia definitiva** en `docs/01_secuencia_mensajes.md`: contrastar el BORRADOR con GSMA V1.1 §3.2.1.3 y verificar que las cuentas Recibe/Emite por nodo coincidan con la tabla objetivo. **Es bloqueante: hasta cerrarla, nadie codifica lógica de mensajes.**
- Probar el `tools/mock_examiner` con un nodo dummy (recibe → 2 notificaciones → reenvía).
- **DoD:** un mensaje atraviesa 2 nodos dummy generando 4 notificaciones bien formadas en el `mock_examiner`, con el timing correcto; secuencia definitiva publicada.

Fase 2 — Implementación individual (*cada uno, días 2–4*)

- Cada integrante escribe el `handle()` de sus 2 nodos siguiendo la ficha de `docs/03_asignacion_nodos.md` y la secuencia.
- Definir los **datos simulados**: perfil de cada UE en el HSS, IP@ por UE, credenciales dummy (acordar entre quien tiene HSS, PGW y los UE).
- Test local: cada nodo contra el `mock_examiner` y "stubs" simples de sus vecinos.
- **DoD:** cada nodo, alimentado con sus mensajes de entrada, emite las notificaciones y los mensajes de salida correctos en aislamiento.

Fase 3 — Integración por tramos (*grupos chicos, días 4–5*)

- **Tramo Attach:** UE → eNB → MME → {HSS, SGW → PGW → PCRF} → ... → Default Bearer (A01–A24).
- **Tramo IMS:** UE → P-CSCF → I-CSCF → S-CSCF → {HSS, TAS} → Subscribe/Notify (B01–B32).
- Probar **primero en localhost** (todos los nodos en una PC con sus puertos), después en LAN.
- **DoD:** cada tramo corre de punta a punta para 1 UE sin intervención manual.

Fase 4 — Integración total y ensayo (*todos, días 5–6*)

- Corrida **end-to-end con 2 UE** (secuencial) en la LAN, con el `mock_examiner` haciendo de examinador.
- Verificar: timing 1 s/1 s, JSON exacto, **TAS reporta ambos UE, S-CSCF emite `fin`**.
- **Ensayo general cronometrado** (la corrida dura varios minutos por los sleeps; es normal).
- **DoD:** corrida completa reproducible; log del `mock_examiner` revisado y correcto.

Fase 5 — Día del examen (*todos*)

- Completar `config/nodes.json` con las **IPs reales** de las laptops y la **IP del PC del examinador**; cargar las identidades IMS que asigne el examinador a cada UE.
 - Seguir el **checklist de arranque** (sección 8).
-

6. Estructura del repositorio

```
SistemasDeHobbins_TeamCamilaFerreira/
├─ README.md           # orientación rápida + índice
├─ PLAN.md / PLAN.pdf  # este plan (fuente y PDF)
├─ docs/
│   ├─ 00_consigna.md   # enunciado reconstruido
│   ├─ 01_secuencia_mensajes.md # BORRADOR de la Figure 3 (contrato a cerrar)
│   ├─ 02_contratos.md  # envelope, notificación, timing
│   └─ 03_asignacion_nodos.md # reparto + fichas por nodo
├─ config/
│   └─ nodes.example.json # plantilla de direccionamiento (copiar a nodes.json)
├─ common/              # librería compartida (Fase 1)
├─ nodes/<12 carpetas>/ # un README por nodo; el código va acá (Fase 3)
├─ tools/mock_examiner/ # examinador de prueba (UDP 55555)
└─ scripts/             # md2pdf.py, build_pdf.ps1, gen_node_readmes.py
```

7. Cómo empezar (setup)

```
# 1. Clonar el repo y entrar
git clone <url-del-repo>
cd SistemasDeHobbins_TeamCamilaFerreira

# 2. Python 3 (probado 3.11) y dependencia para generar el PDF
python --version
pip install markdown pygments

# 3. Copiar el registro de nodos y completarlo
copy config\nodes.example.json config\nodes.json # luego editar IPs y apellidos

# 4. Levantar el examinador de prueba (en una terminal aparte)
python tools\mock_examiner\mock_examiner.py
```

`config/nodes.json` **no** se versiona con IPs reales del examen (es por entorno); sí se versiona la plantilla `nodes.example.json`.

8. Checklist del día del examen

1. [] Todas las laptops en la **misma LAN**; anotar la IP de cada una.
2. [] Completar `config/nodes.json`: IP/puerto de los 12 nodos + **IP del examinador**.
3. [] Cargar las **identidades IMS** (`tel:+598...`) que asigne el examinador a UE1 y UE2.
4. [] Verificar `apellido` correcto en cada nodo.
5. [] **Prueba de humo**: cada nodo manda 1 notificación de test y el examinador la ve.

6. [] Orden de arranque: levantar **primero los nodos de red** (HSS, MME, CSCFs, TAS...) y **por último los UE**, que son los que inician el flujo.
 7. [] Correr UE1 (Attach + Registration completo) y verificar; luego UE2.
 8. [] Confirmar: TAS reportó ambos UE y S-CSCF mandó `{"message": "fin"}`.
-

9. Riesgos y cosas a confirmar con el examinador

- **Secuencia exacta:** el BORRADOR de la Figure 3 debe cerrarse contra GSMA V1.1; los puntos ambiguos son el bloque **Authentication/Security, UDR/UDA** (¿TAS⇒HSS?), **AAR/AAA (Rx)** y el camino del **SUBSCRIBE/NOTIFY**. (Ver §"Puntos a reconciliar" en [docs/01_secuencia_mensajes.md](#).)
 - **Mensaje que origina un nodo sin recibir** (primer `RRC` del UE): ¿lleva notificación `a_enviar`? Confirmar.
 - **2 UE:** ¿secuencial (lo asumido) o en paralelo?
 - **Nombres de las claves JSON** de la notificación: los definimos nosotros; confirmar que el formato sea aceptable.
 - **Pérdida de paquetes UDP:** en LAN/localhost es muy baja, pero si aparece, evaluar reintentos/ACK simples a nivel aplicación.
 - **Firewall de Windows:** puede bloquear UDP entrante; permitir Python en la red privada.
-

10. Glosario rápido

UE equipo de usuario · **eNB** estación base LTE · **MME** gestión de movilidad · **SGW/PGW** gateways de datos · **PCRF** políticas/QoS · **HSS** base de suscriptores · **P/I/S-CSCF** servidores de control de sesión SIP del IMS · **TAS** servidor de aplicación de telefonía · **Attach** enganche a la red móvil · **IMS Registration** registro SIP · **Default Bearer** portador de datos para señalización IMS (QCI 5).